

Augmented Lagrangian Predictive Coding

PRIVATE SPEAKER GUIDE

27-minute talk + 3-minute discussion
20 main slides + 6 optional backup slides

Use Talk track during the allotted time. From first principles / preparation is additional rehearsal and reference material. This guide contains approximately 8,400 words; it is not a script to read in full during a half-hour slot.

The notes are also embedded in the PowerPoint notes pane. The separate audience PDF contains slides only. Sources are linked after each slide's notes, with a pinned FabricPC commit for reproducibility.

Source distinction

Paper findings are attributed to the supplied Seely-Gould preprint. The scalar example is independently computed. Repository statements come from code and documentation inspection, not new library benchmarks. The FabricPC integration and evaluation sequence are proposals.

Running order

- 01 00:00–00:30 Augmented Lagrangian Predictive Coding
- 02 00:30–02:00 First: what are we trying to learn?
- 03 02:00–03:30 Backpropagation assigns credit by the chain rule
- 04 03:30–05:00 Standard PC lets hidden states move
- 05 05:00–06:30 A finite penalty changes the equilibrium
- 06 06:30–08:00 ePC optimizes prediction errors directly
- 07 08:00–09:30 ePC can be BP-like before it equilibrates
- 08 09:30–11:00 Add a multiplier that remembers violations
- 09 11:00–13:00 One batch: relax, accumulate, then learn
- 10 13:00–14:30 Replace the residual credit with a composite
- 11 14:30–16:00 Why the endpoint gives the BP gradient
- 12 16:00–17:30 A scalar example makes the difference visible
- 13 17:30–18:30 The paper observes faster credit propagation
- 14 18:30–20:00 The headline result: the deep-narrow gap closes
- 15 20:00–21:30 Promising mechanism; limited practical evidence
- 16 21:30–23:00 Three mechanisms, three different tradeoffs
- 17 23:00–24:30 FabricPC needs a coordinated extension
- 18 24:30–26:00 Evaluate correctness before model-scale claims
- 19 26:00–27:00 Recommendation: test PC-ALM as a research method
- 20 27:00–30:00 Discussion: what should the first experiment prove?
- 21 Optional backup Notation and conventions
- 22 Optional backup Derive the scalar example by hand
- 23 Optional backup Why ePC changes dynamics but preserves the objective
- 24 Optional backup Linear stability is a property of the joint update
- 25 Optional backup Integration details that affect the algorithm
- 26 Optional backup Sources and reproducibility

01 Augmented Lagrangian Predictive Coding

00:00–00:30

Talk track

Today I want to explain how a network can compute useful learning signals by repeatedly updating local variables. We will build standard predictive coding from a familiar supervised network, show what FabricPC's error predictive coding changes, and then derive the augmented Lagrangian mechanism. The question for our team is whether this mechanism is worth adding to FabricPC. The paper gives a promising answer for local credit assignment, but it does not yet establish a practical advantage over our ePC implementation.

From first principles / preparation

This is a 30-minute slot: 27 minutes of prepared material followed by 3 minutes of discussion. Slides 21–26 are backup. The talk track is a concise speaking guide; these preparation paragraphs contain the extra derivations needed to answer questions. Do not attempt to read all preparation text aloud.

The paper is Jeffrey Seely and Julian Gould's Augmented Lagrangian Predictive Coding, arXiv:2605.31022v1, dated 29 May 2026. We analyze that supplied version, rather than silently substituting another version. PC-ALM is the authors' name for their finite-inference algorithm. ALM means augmented Lagrangian method. ePC means error-parameterized predictive coding; it is not equilibrium propagation, usually abbreviated EP.

The comparison is tied to FabricPC commit 8406e6a838442391fd3089958e1a2c1b44c57eda, retrieved 22 September 2026. Treat engineering recommendations as proposals. The paper figures are reported evidence; the scalar example later is our own exact calculation. No new FabricPC training benchmark was run for this presentation.

Transition

Start with the ordinary supervised learning problem.

Sources

[P] [Paper v1](#) · [R] [ePC solver](#)

02 First: what are we trying to learn?

00:30–02:00

Talk track

An input x enters a sequence of layers. Each layer applies a weight matrix and a nonlinearity to produce an activation h . The final layer produces a prediction, which we compare with a target y . A loss is a scalar that measures the discrepancy. Learning means changing the weights to reduce that loss across examples.

There are two kinds of variables here. Weights are shared across training examples and are what we eventually deploy. Hidden activations belong to the current example. In an ordinary feedforward network, the activations are determined by the weights and input. Predictive coding temporarily treats those activations as adjustable variables. That distinction is the starting point for everything that follows.

From first principles / preparation

Notation used throughout: L is the number of weight-bearing layers, including the output readout; i indexes a hidden layer from 1 to $L-1$. W_i is layer i 's weight matrix, and θ is the collection of all weight matrices. h_i is layer i 's activation vector for one example. $h_0 = x$ is the clamped input. σ is the elementwise activation function, such as identity, tanh, or ReLU. The output prediction is $\hat{y} = W_L h_{L-1}$; the paper uses a linear readout. ℓ is the supervised loss, here $\ell = \frac{1}{2} \|y - \hat{y}\|^2$. The squared Euclidean norm is the sum of squared component differences. The factor $\frac{1}{2}$ simplifies derivatives.

For a matrix W_i with dimensions n_i by n_{i-1} , h_{i-1} has n_{i-1} components and h_i has n_i components. Biases are omitted to match the paper. A nonlinearity applies a scalar function to each component of $W_i h_{i-1}$. The paper's experiments additionally use residual connections and width/depth scaling; the simple chain is our teaching model, not a claim that the benchmark architecture is a plain chain.

A training gradient is the vector or matrix of partial derivatives of a scalar objective. A negative gradient step changes each weight in the direction that locally reduces that objective, for a sufficiently small step. It need not reduce test error on every update. Throughout the inference derivations, the weights are fixed.

Transition

Backpropagation answers how a distant weight affects the final loss.

Sources

[P] Paper v1

03 Backpropagation assigns credit by the chain rule

02:00–03:30

Talk track

Suppose we slightly change one hidden activation. The next layer changes, then the next, and eventually the output loss changes. The chain rule multiplies these sensitivities. Backpropagation evaluates them efficiently in reverse order.

Call the resulting hidden-activation derivative δ . At the last hidden layer, δ is the output error pulled back through the readout matrix. At earlier layers, the next layer's Jacobian transposed pulls δ backward again. A Jacobian is just a table of local derivatives. Once we know δ , the weight gradient combines it with the input that produced that activation. These are the credit signals PC-ALM aims to obtain through repeated local dynamics.

From first principles / preparation

Define δ_i as the total derivative $d\ell/dh_i$ of the composed downstream network, evaluated at forward-pass activations. $J_{i+1} = \partial h_{i+1} / \partial h_i$ is the Jacobian of the adjacent layer map. For $h_{i+1} = \sigma(W_{i+1} h_i)$, it equals $\text{diag}(\sigma'(W_{i+1} h_i)) W_{i+1}$. A superscript T denotes transpose. The chain rule gives $\delta_i = J_{i+1}^T \delta_{i+1}$, with boundary $\delta_{L-1} = W_L^T (\hat{y} - y)$.

For a hidden weight matrix, $\partial \ell / \partial W_i = (\delta_i \odot \sigma'(W_i h_{i-1})) h_{i-1}^T$, where $\odot$ denotes componentwise multiplication. The outer product pairs a postsynaptic sensitivity with a presynaptic activation. A weight update subtracts the learning rate times that gradient.

This equation is spatially local once the correct sensitivity arrives. The distinction is the computation that supplies it: standard reverse-mode differentiation traverses the whole composed graph in dependency order and uses saved forward information. It is therefore too strong to say BP's arithmetic is intrinsically nonlocal. PC-ALM's claim concerns obtaining the signals via iterated adjacent-layer interactions instead of a conventional global reverse pass. Neither local derivatives nor the absence of a global reverse pass alone prove biological realism.

Transition

Predictive coding replaces the exact forward constraints with local mismatch costs.

Sources

[P] [Paper v1](#)

04 Standard PC lets hidden states move

03:30–05:00

Talk track

Now let each hidden layer hold an adjustable state h . Its upstream neighbor predicts what that state should be. Their difference is the residual r . Standard PC minimizes the output loss plus a quadratic cost for all hidden residuals.

Training has two phases. First, hold the weights fixed and adjust the hidden states for T inference steps. Then hold those final states fixed and update the weights using local energy gradients. An interior state appears in its own mismatch and its successor's mismatch, so its gradient only needs adjacent layers. The output supervision initially moves states near the readout; that disturbance reaches earlier layers through repeated updates.

From first principles / preparation

The residual is $r_i = h_i - \sigma(W_i h_{i-1})$. A residual is a vector with the same shape as the activation. It is not necessarily the output label error. $\rho > 0$ is the shared hidden-constraint penalty strength. The standard PC energy is $F_{PC} = \ell + (\rho/2) \sum_i \|r_i\|^2$. T is the number of inference steps, η_h is the activity step size, and η_θ is the weight learning rate. All sums over hidden constraints run from $i=1$ to $L-1$ unless stated otherwise.

An interior hidden gradient is $\partial F_{PC} / \partial h_i = \rho r_i - J_{i+1}^T \rho r_{i+1}$. The first term pulls the state toward its own prediction. The second term lets it change to improve the successor's prediction. The last hidden layer instead receives $\rho r_{L-1} + W_L^T (\hat{y} - y)$. Input x is clamped and does not undergo this update.

After inference, the hidden-weight gradient is $-(\rho r_i \odot \sigma'(W_i h_{i-1})) h_{i-1}^T$. Evaluate it at final inferred states and treat those states as fixed. Differentiating through the full history of inference would define a different learning algorithm. In FabricPC, InferenceSGD implements state updates using node-local gradients. This description assumes plain gradient descent with no latent decay, clipping, or non-gradient scaling. Those choices can change the dynamics and must be stated in a comparison.

The word inference here means training-time state relaxation. It should not be confused with an ordinary production forward pass.

Transition

Fast convergence and the right equilibrium are separate questions.

Sources

[P] [Paper v1](#) · [I] [sPC solver](#)

05 A finite penalty changes the equilibrium

05:00–06:30

Talk track

The penalty does not force the residuals to vanish. It permits a compromise: move the hidden state away from its forward value if that sufficiently improves the supervised loss. Even perfectly converged PC can therefore produce a different weight update from backpropagation.

Consider one hidden scalar. Let the input and target both be one, the first weight be 0.2, and the second weight be one. The forward hidden value is 0.2. With penalty one, PC instead settles at 0.6: halfway between the upstream prediction and the value preferred by the output loss. Solving this problem faster does not change that endpoint. Raising the penalty reduces the mismatch, but can make the numerical solve stiff.

From first principles / preparation

For this example, h is a single adjustable hidden activation, w_1 and w_2 are the two scalar weights, and $x=y=1$. The forward output is $w_2 w_1 x = 0.2$. With $\rho=1$, $F_{PC}(h) = \frac{1}{2}(1-h)^2 + \frac{1}{2}(h-0.2)^2$. Its derivative is $2h-1.2$, so $h^*=0.6$. The second derivative is $2>0$, proving this is the unique minimum. The hidden residual is 0.4 and the output target-minus-prediction residual is also 0.4.

More generally, $h^* = (w_2 y + \rho w_1 x) / (w_2^2 + \rho)$. As ρ grows, h^* approaches $w_1 x$. However, the activity curvature grows with ρ , limiting an explicit gradient descent step size. This motivates an augmented Lagrangian: a finite quadratic penalty together with a multiplier can enforce feasibility at an appropriate fixed point.

The scalar weight gradients at the PC equilibrium are $\partial F_{PC} / \partial w_1 = -0.4$ and $\partial F_{PC} / \partial w_2 = -0.24$. BP on the original forward network gives -0.8 and -0.16 . The pair differs in direction, not just overall magnitude. The state-relaxed objective is legitimate if those soft internal constraints are part of the intended model. Calling the difference a defect only makes sense when matching the feedforward supervised objective is the goal.

This presentation's sPC means the finite-penalty, state-relaxation baseline used here. Other PC schedules and limiting constructions can reproduce BP under additional assumptions. We are not making a claim about every algorithm called predictive coding.

Transition

FabricPC's ePC changes how this same energy is optimized.

Sources

[P] [Paper v1](#)

06 ePC optimizes prediction errors directly

06:30–08:00

Talk track

In ePC, each hidden error ε becomes the variable we optimize. Given those errors, we rebuild the hidden states in forward order: prediction plus error. Changing an early error changes every downstream derived state. Differentiating the resulting energy with respect to the errors can therefore transmit the output signal across the whole network in one reverse pass.

This is what FabricPC's EPCInference does. It can greatly reduce the number of inference iterations needed on a deep graph, while retaining the existing local weight-learning step. The important distinction is that this inference gradient uses a full-network reverse pass. There is no extra multiplier, and the underlying PC energy has not acquired a new constraint term.

From first principles / preparation

Use ε_i for an independently optimized ePC error coordinate. Its numerical value equals $r_i = h_i - \sigma(W_i, h_{i-1})$ after deriving states, but its role differs: sPC changes h and recomputes residuals; ePC changes ε and reconstructs h . The recurrence is $h_i(\varepsilon) = \sigma(W_i, h_{i-1}(\varepsilon)) + \varepsilon_i$. Define $E(\varepsilon, \theta) = F_{PC}(h(\varepsilon), \theta)$. The update is $\varepsilon \leftarrow \varepsilon - \eta_\varepsilon \nabla_{\varepsilon} E$, where η_ε is the error-coordinate step size.

On a directed acyclic graph, fixed input clamps and the triangular forward recurrence give a one-to-one correspondence between free hidden states and errors. Both directions can be computed in topological order. The diagonal blocks of the state/error Jacobian are identities, so it is invertible. A smooth invertible coordinate change preserves stationary points and minima, although it changes gradients, conditioning, basins visited by discrete trajectories, and convergence speed. It does not guarantee that two nonlinear solvers land in the same minimum.

FabricPC implements the full error gradient through `errorenergy`, `derivestates`, and `jax.value_andgrad`. Its final derived states feed the separate local weight-gradient routine. On cyclic graphs, the implementation differentiates a finite unrolled computation with carried states; the exact acyclic coordinate-equivalence argument no longer applies. Also, its nonlinear μ PC state-gradient preconditioning can shift the sPC fixed point, as explicitly covered by a repository test. Thus ‘same equilibrium’ on the slide means the same acyclic objective with compatible gradient dynamics, not unrestricted equivalence of every library configuration.

Transition

This also explains why an ePC result must be labeled by its inference regime.

Sources

[E] [ePC paper](#) · [R] [ePC solver](#) · [T] [ePC tests](#)

07 ePC can be BP-like before it equilibrates

08:00–09:30

Talk track

At forward initialization all hidden errors are zero. The first error-coordinate gradient is then exactly the BP activation derivative. A small ePC step stores a scaled copy of that derivative in the hidden errors. Local weight updates built from those errors are approximately BP-like, with an important scaling difference between hidden layers and the readout.

If we keep relaxing ePC until it converges, we instead approach the PC energy's equilibrium. PC-ALM is designed to reach the BP-consistent endpoint through its augmented constraints. So more inference does not have the same interpretation for both methods. A fair FabricPC comparison needs both the small-step ePC regime and ePC near equilibrium.

From first principles / preparation

At $\epsilon=0$, the derivative of the quadratic hidden penalty is zero. Since adding ϵ_i perturbs h_i additively, the remaining derivative $\partial E / \partial \epsilon_i$ equals δ_i , the BP derivative at the forward states. One exact gradient step gives $\epsilon_i^1 = -\eta_\epsilon \delta_i$. With unit hidden precision, the local hidden-weight gradient therefore has leading term η_ϵ times the BP weight gradient. The readout retains an order-one supervised gradient. With general hidden penalty ρ , the leading hidden scaling is $\rho \eta_\epsilon$.

After updating errors, FabricPC re-derives hidden states before learning. An upstream perturbation changes the input and possibly the activation derivative used by a downstream node. Therefore the final weight gradients are generally only first-order BP-like for small η_ϵ , rather than exactly BP at arbitrary finite step size. A layer fed exclusively by clamped inputs has a stronger exact relationship for its hidden gradient.

Adam approximately normalizes a persistent positive gradient scale when its denominator's stabilizing constant is negligible and the scaling remains consistent; SGD does not. Optimizer behavior is part of the algorithm comparison, not evidence that all finite-step ePC updates equal BP.

For a positive-curvature quadratic error energy with Hessian H_ϵ , a sufficient full-spectrum gradient-descent bound is $0 < \eta_\epsilon < 2 / \lambda_{\max}(H_\epsilon)$. Here $\lambda_{\max}$ denotes a Hessian eigenvalue, not a Lagrange multiplier. The relevant curvature can change during training. FabricPC provides `epsilon_spectrum` and `RegimeProbe` to monitor the regime. Negative curvature requires additional interpretation; the simple contraction statement applies only to positive modes. The repository's long-run examples motivate measuring this rather than transferring default rates between graphs.

Transition

Now we add a new variable with a different job from an optimized error.

Sources

[R] ePC solver · [G] ePC guide · [D] Stability report

08 Add a multiplier that remembers violations

09:30–11:00

Talk track

Return to the original network equations: every hidden state should equal its upstream prediction. We want the output loss to improve while enforcing those equations. Attach a multiplier λ to each hidden residual. It has one component per activation and per example. Add the dot product $\lambda \cdot r$ to the PC energy.

The hidden states descend this augmented objective. The multipliers ascend it, accumulating the residual after the state update. A persistent positive violation increases λ ; a negative violation decreases it. The quadratic term supplies curvature, while the accumulated multiplier can continue carrying a learning signal even when the instantaneous residual becomes zero. Setting all multipliers permanently to zero recovers standard PC.

From first principles / preparation

The augmented Lagrangian is $L_p(h, \theta, \lambda) = \ell(h, \theta) + \sum_i \lambda_i^T r_i + (\rho/2) \sum_i \|r_i\|^2$. λ_i is the hidden-layer multiplier vector, with the same activation shape as h_i . It is sample-specific inference state, not a shared trainable weight and not a momentum estimate of the weight gradient. In a minibatch it also has a batch axis.

Why ascent? For an equality constraint $r_i=0$, the derivative of L_p with respect to λ_i is exactly r_i . Updating λ_i in the ascent direction applies persistent pressure against an unresolved violation. A primal variable, here h , is a variable in the original constrained problem; a dual variable, here λ , is attached to a constraint. At an appropriate stationary feasible point, both the residual and the primal derivative vanish. The multipliers themselves need not vanish.

For fixed λ , completing the square gives $\lambda_i^T r_i + (\rho/2) \|r_i\|^2 = (\rho/2) \|r_i + \lambda_i/\rho\|^2 - \|\lambda_i\|^2/(2\rho)$. Since $r_i = h_i - \text{prediction}_i$, the effective prediction target is $\text{prediction}_i - \lambda_i/\rho$. Keep the parentheses: $h_i - (\text{prediction}_i - \lambda_i/\rho)$ equals $r_i + \lambda_i/\rho$. This checks the sign of the implementation.

Classical method of multipliers approximately or exactly solves the primal subproblem before a dual update. PC-ALM is the paper's finite-budget variant that substitutes one primal gradient step. It does not inherit every classical convergence theorem. It is also not ADMM: no alternating exact block minimization is used in Algorithm 1.

Transition

The exact order of the state and multiplier updates matters.

Sources

[P] [Paper v1](#)

09 One batch: relax, accumulate, then learn

11:00–13:00

Talk track

Initialize the hidden states with a forward pass and reset the hidden multipliers to zero for the new examples. Hold the weights fixed. Repeatedly take one activity descent step, recompute residuals at those new activities, and add α times those residuals to the multipliers.

The paper repeats that pair $T-1$ times, then takes one final activity step without a final dual update. Only then are the weights updated using the augmented Lagrangian's local derivatives. Thus the first activity step matches PC, and $T=1$ gives the same update as the matched one-step PC baseline. α is a separate dual rate; it is not automatically equal to the penalty ρ . The multiplier integrates errors within this inference solve and is reset for the next batch.

From first principles / preparation

An exact implementation should distinguish old and new state. At inference iteration t , compute all hidden activity gradients from the same h^t and λ^t . Apply $h^{t+1} = h^t - \eta_h \nabla_{h_{LP}}(h^t, \theta, \lambda^t)$ simultaneously across free hidden variables. Recompute $r_i(h^{t+1}, \theta)$, using the newly updated state of both a node and its upstream neighbors. Then set $\lambda_i^{t+1} = \lambda_i^t + \alpha r_i(h^{t+1}, \theta)$. A sequential layer sweep that exposes freshly updated states to later primal gradients is a different discretization.

After $T-1$ such cycles, take the final primal step and recompute residuals again for learning. Its composite signal uses the residual at the final h and the λ from the last dual update. The paper's endpoint theorem is about repeated primal-dual dynamics at fixed weights; finite T is the practical algorithm, not an assertion of exact convergence.

Classical method of multipliers uses dual increment ρ after a settled primal solve. The finite-step algorithm chooses α independently because its primal solve is intentionally incomplete. η_h is the activity rate, α is the dual rate, and η_θ is the weight rate. They act on different variables. The paper uses a shared penalty and no hidden bias terms in its experiments.

Keep the output objective separate: the hidden equations are constraints, while the output target discrepancy is the supervised loss. Attaching a multiplier to a hard output clamp would impose a different constraint and could force impossible exact target fitting at fixed weights. Minibatch gradients are averaged after computing sample-local contributions. Do not average the residual across the batch before updating per-example multipliers.

Transition

The code change is governed by one composite local signal.

Sources

[P] Paper v1

10 Replace the residual credit with a composite

13:00–14:30

Talk track

Define the composite credit c as the multiplier plus the penalty-scaled residual. For an interior hidden layer, the activity gradient is its own c minus its successor's c pulled back through the adjacent Jacobian. This has the same local shape as the PC update.

The hidden-weight gradient also uses c : combine it with the local activation derivative and incoming activity. Using multipliers only during inference would miss half the method. Once residuals become small, the multiplier remains the dominant learning signal. With the residual convention used here, c has the opposite sign to the BP activation derivative at the endpoint; the minus sign in the weight formula recovers the correct gradient.

From first principles / preparation

Define $c_i = \lambda_i + \rho r_i$. The interior derivative is $\partial L_\rho / \partial h_i = c_i - J_{i+1}^T c_{i+1}$. The boundary derivative at h_{L-1} is $c_{L-1} + W_L^T (\hat{y} - y)$. Thus every term uses either the layer's own state or an adjacent node's information. A general acyclic graph replaces the single successor contribution by a sum over outgoing edges.

For hidden weights, $\partial L_\rho / \partial W_i = -(c_i \odot \sigma'(W_i h_{i-1})) h_{i-1}^T$. For the readout, $\partial L_\rho / \partial W_L = (\hat{y} - y) h_{L-1}^T$. Holding h and λ fixed during this derivative is essential. At finite inference, hidden inputs are inferred rather than necessarily feedforward, so c alignment alone is not enough to prove equality of the full weight gradient.

The residual remains r_i . It should not be overwritten by c_i in a state object whose documented invariant is $\text{error} = z_{\text{latent}} - z_{\text{mu}}$. A diagnostic reporting c as reconstruction error would conceal feasibility. Likewise, substituting a pseudo-error in FabricPC's final state is insufficient if the learning routine recomputes the node's prediction and energy. The augmented term must participate in the actual local derivative path.

For complex nodes, derive gradients through the local prediction function and the scalar local augmented term $\lambda_i^T r_i + (\rho/2) \|r_i\|^2$. This preserves the chain rule within that node, including any local bias parameters, without coding a matrix-only shortcut. The result is an engineering extension beyond the paper's tested bias-free residual MLPs.

Transition

At equilibrium, feasibility and stationarity force the BP recursion.

Sources

[P] [Paper v1](#) · [W] [Local learning](#)

11 Why the endpoint gives the BP gradient

14:30–16:00

Talk track

At a fixed point with a positive dual rate, the multipliers stop changing only if every hidden residual is zero. So the hidden states again equal the ordinary forward pass. At the same time, the activity gradients must vanish. Starting at the output boundary, that condition says the last multiplier equals minus the BP derivative. Moving backward, the same adjacent Jacobians reproduce the BP chain rule at every layer.

Therefore the local weight derivatives equal BP at that endpoint. Two claims must be kept separate: what a feasible stationary endpoint would compute, and whether the actual iteration reaches it. The paper proves convergence for its stable linear setting. Nonlinear finite-budget performance is supported by experiments, not by a universal convergence theorem.

From first principles / preparation

A Karush-Kuhn-Tucker, or KKT, point satisfies appropriate first-order constrained optimality conditions. Here we use the fixed-weight inner activity problem. The conditions needed for the argument are primal feasibility $r_i=0$ and stationarity $\partial L_p / \partial h_i = 0$. Do not add $\partial L_p / \partial W_i = 0$: weights are fixed during this inner problem, and their nonzero derivatives are the learning signal.

At feasibility, $c_i = \lambda_i$. The output boundary gives $\lambda_{L-1} = -W_L^T(\hat{y} - y) = -\delta_{L-1}$. Interior stationarity gives $\lambda_i = J_{i+1}^T \lambda_{i+1}$. By induction, $\lambda_i = -\delta_i$ at all hidden layers. Substituting this into the hidden-weight formula gives $+(\delta_i \odot \sigma'(W_i h_{i-1})) h_{i-1}^T$, exactly the usual BP derivative. The minus sign follows from choosing residual state-minus-prediction; reversing the residual convention reverses multiplier signs.

The endpoint identity extends to differentiable nonlinear feedforward maps and acyclic graphs with skips whenever these conditions hold. At ReLU kinks, interpret derivatives with a consistent subgradient convention. This algebra is not a nonlinear convergence guarantee.

The linear theorem fixes weights, assumes identity activations and positive rates and penalty, and requires the spectral radius of the full joint iteration matrix to be below one. Under those assumptions, the affine dynamics converge to a unique fixed point. The authors' explicit proof and wave analysis omit biases. Accuracy matching BP after a fixed number of steps does not prove that every per-example weight gradient is exactly equal to BP.

Transition

The scalar example makes the residual-versus-multiplier distinction visible.

Sources

[P] [Paper v1](#)

12 A scalar example makes the difference visible

16:00–17:30

Talk track

Here are independently computed traces for the two-weight network. PC moves h from 0.2 to 0.6 and stays there. PC-ALM first moves the state in response to the output loss, then the accumulated multiplier brings it back toward 0.2. The multiplier settles at 0.8, which stores the missing credit.

The endpoint weight-gradient pair is revealing. PC gives minus 0.4 and minus 0.24. PC-ALM gives minus 0.8 and minus 0.16, exactly the BP pair. The signs tell us how the loss changes with the weights; the actual learning update subtracts these values times the learning rate. These traces are a teaching calculation, not a reproduced Fashion-MNIST benchmark.

From first principles / preparation

Use $x=y=1$, $w_1=0.2$, $w_2=1$, $p=1$, $\eta_h=0.2$, and $\alpha=0.5$. The scalar augmented objective is $\frac{1}{2}(1-h)^2 + \lambda(h-0.2) + \frac{1}{2}(h-0.2)^2$. Its activity derivative is $2h-1.2+\lambda$. A full primal-dual cycle is $h_{\text{new}} = h - 0.2(2h-1.2+\lambda)$, followed by $\lambda_{\text{new}} = \lambda + 0.5(h_{\text{new}} - 0.2)$.

Starting from $h=0.2$ and $\lambda=0$, the first PC and PC-ALM activity update both give $h=0.36$. The first ALM dual update gives $\lambda=0.08$. The next ALM activity value is 0.44, while PC reaches 0.456. The joint ALM fixed point is $h=0.2$, $\lambda=0.8$. The plotted activation curve may oscillate before settling; its decay is verified numerically with this stable rate choice.

At PC equilibrium, $r=0.4$: gradient $w_1 = -r \cdot x = -0.4$; gradient $w_2 = (w_2 h - y) h = -0.24$. At the ALM endpoint, $r=0$ and $c=\lambda=0.8$: gradient $w_1 = -c \cdot x = -0.8$; gradient $w_2 = (0.2 - 1) \cdot 0.2 = -0.16$. These match direct differentiation of $\frac{1}{2}(1 - w_2 w_1)^2$.

The plots show the repeated primal-dual recurrence to illustrate its equilibrium. A finite Algorithm 1 training call ends with an additional primal-only step; that detail is respected in the presentation's algorithm and separate numerical check. The one-hidden-variable ePC coordinate transform is just a translation, $h=0.2+\epsilon$. With matching activity/error rates its trajectory therefore coincides with PC in this scalar example. This example demonstrates different objectives, not an ePC speed advantage or disadvantage.

Transition

In deep networks, the paper reports a different pattern of credit propagation.

Sources

[P] [Paper v1](#)

13 The paper observes faster credit propagation

17:30–18:30

Talk track

This is the paper's Figure 3, for an initialized 64-layer ReLU residual MLP of width 32. Look first at the left column. Standard PC concentrates the credit close to the supervised end. With the multiplier update, the composite signal reaches earlier layers more effectively. The middle column separates the instantaneous residual from the accumulated multiplier.

The authors call the propagation ballistic. In their linear long-wavelength analysis, the characteristic arrival time scales roughly with depth divided by the square root of the product of activity and dual rates. This describes a useful wavefront, not instant communication across layers and not the time to arbitrary convergence accuracy.

From first principles / preparation

Orient the audience: layer index increases from the input side at the left toward the output side at the right. Output supervision first affects the rightmost hidden layers. The top row is PC; the lower rows are PC-ALM at two dual rates. Each trace corresponds to a different inference time. The first column plots composite-credit magnitude, the middle residual and multiplier norms using the figure's distinct axis scales, and the final column plots alignment with BP hidden credit.

In a linear homogeneous or near-identity setting, ordinary state gradient dynamics resemble diffusion, with distance traveled scaling approximately as the square root of time. The joint primal-dual recurrence can have complex eigenvalues and support wave-like propagation, with distance proportional to time over the relevant regime. Appendix D develops this approximation. The indicative arrival time is $t \approx L/\sqrt{\alpha\eta_h}$, where t counts inference steps.

Keep three qualifications. The argument depends on the operator and parameterization; it is not a universal bound for any neural network. The 'wavefront' marks propagation or an alignment inflection, not full settling to a prescribed tolerance. Increasing α can accelerate the front but also violate stability. Figure 3 is a single initialized sample, not a distribution of trained-network outcomes. The approximate uniformity of the plotted credit norms does not imply equal learning signal at every layer of all networks, or elimination of all ordinary Jacobian-related vanishing gradients.

Transition

The classification experiments test whether those signals lead to useful learning.

Sources

[P] [Paper v1](#)

14 The headline result: the deep-narrow gap closes

18:30–20:00

Talk track

This is Figure 2 from the paper. Width runs horizontally and depth vertically. The top plots compare BP, PC-ALM, and PC. The lower plots show the accuracy difference between PC-ALM and PC. The main pattern is the improvement in deep, narrow networks, where ordinary PC is weakest.

The authors report that PC-ALM matches BP performance across their tested grid when given $T=2L$ activity steps. The grid spans widths and depths from 8 to 128 and identity, tanh, and ReLU activations. MNIST gives a similar pattern in the appendix. This is encouraging evidence for the mechanism, but the experiment is one epoch of training on small image datasets. There is no ePC baseline in this comparison.

From first principles / preparation

Give the audience a few seconds to inspect the plots rather than narrating all curves. In the lower row, positive warm colors mean PC-ALM achieved higher test accuracy than PC. The paper presents categorical claims about closing the PC-BP gap; we do not extract invented exact percentages from the small plotted markers. Depth L counts the network's weight-bearing layers and N is hidden width.

The experiment uses residual multilayer perceptrons and the authors' mean-field parameterization following prior PC work. Width and depth each take values $\{8,16,32,64,128\}$. The paper fixes an activity rate from an empirical constraint-operator estimate for each architecture and dataset. Its appendix specifies batch size 64 and one epoch on MNIST and Fashion-MNIST. The MNIST grid caption explicitly describes three seeds; the separate parameterization probe is explicitly single-seed. Avoid implying that all panels have extensive independent replication or confidence intervals.

At $T=L$, the paper reports partial recovery of the PC-BP gap; at $T=2L$ it reports matching BP across the tested settings. 'Matches' is their empirical performance conclusion, not a statistical equivalence test stated in this talk. Matched T means the two PC methods have equal counts of activity updates, with added multiplier work for PC-ALM. BP has a different computation structure. The paper does not benchmark FabricPC, ePC, convolutions, attention layers, or long-horizon training. An engineering decision must account for those missing comparisons.

Transition

The limitations determine how strongly we should act on that result.

Sources

[P] [Paper v1](#)

15 Promising mechanism; limited practical evidence

20:00–21:30

Talk track

The potential advantage is specific: preserve adjacent-layer inference while reducing the finite-penalty discrepancy from BP. The costs include one extra activation-shaped multiplier, residual recomputation, an additional rate to tune, and possible oscillatory instability. Finite inference can still leave substantial gradient error.

The paper's strongest theory is linear and conditional on stability. Its nonlinear evidence covers residual MLPs, two small datasets, and one epoch. Its compute comparison is against widening PC to obtain BP-aligned credit. It does not establish a tenfold speedup over BP or ePC. For FabricPC we need actual wall-clock time, memory, and long-run stability, not just equal iteration counts.

From first principles / preparation

The paper says PC-ALM doubles activation memory in an idealized state accounting: hidden activations h plus equally sized multipliers λ . FabricPC already stores z_{latent} , z_{mu} , error, energy, and gradient accumulators, and its automatic differentiation may keep intermediate tensors. Adding λ is therefore not a claim that total process or accelerator memory will double. The exact increment depends on implementation, batch shape, derivative buffers, optimizer state, and sharding.

The dominant dense-layer work remains roughly proportional to $T L N^2$ per example for iterative PC variants, but constants, device utilization, synchronization, and critical paths matter. A local mathematical update does not automatically execute in parallel on the current software backend. FabricPC ePC incurs a full derived-forward/reverse traversal per error step; it may need fewer steps. There is no basis here for a wall-clock ordering between ePC and ALM.

Figure 8 and Appendix E compare fixed-width ALM with widening PC to reach a hidden-credit cosine target. The reported order-of-magnitude example is a particular $L=32$ ReLU/Fashion-MNIST setting. The asymptotic cost discussion assumes a fixed PC inference budget for its main comparison and a stable ALM spectral gap. It is not a measured general speedup against BP.

Ordinary continuous-time PC gradient flow has no oscillatory eigenmodes near a positive quadratic minimum. Explicit discrete gradient descent can still alternate signs or overshoot when eigenvalues are negative but inside the unit circle. Therefore do not say 'PC can never oscillate'; the distinctive ALM phenomenon is coupled primal-dual modes with complex eigenvalues. In nonlinear training, monitor state residuals, stationarity, gradients, and stability rather than demanding monotone augmented-Lagrangian values during dual ascent.

Transition

Here is the mechanism-level comparison in one place.

Sources

[P] [Paper v1](#) · [R] [ePC solver](#)

16 Three mechanisms, three different tradeoffs

21:30–23:00

Talk track

Read this table across each row. Standard PC optimizes hidden states in the PC energy with local inference. ePC optimizes errors in that same energy, using global differentiation to change how quickly information reaches those variables. PC-ALM evolves hidden states and multipliers for the constrained feedforward problem, with local primal and dual steps.

At compatible equilibria, sPC and ePC solve the relaxed PC problem. A feasible stationary ALM endpoint yields BP weight gradients. Each has a plausible role: sPC for the soft-constraint model and local graph dynamics, ePC for fast digital inference, and ALM for testing BP-like learning through local dynamics. The paper does not tell us which is the best overall FabricPC default.

From first principles / preparation

The table is intentionally conditional. The sPC/ePC equilibrium correspondence assumes an acyclic error/state change of coordinates and true descent on the same energy. Their trajectories need not visit the same nonlinear minimum. FabricPC's cyclic unrolling and nonlinear μ PC gradient preconditioning are concrete exceptions to unqualified equivalence.

For sPC, advantages include a simple scalar energy, adjacent-node derivatives, and a natural interpretation when hidden mismatches are legitimate latent uncertainty or soft constraints. Disadvantages include slow propagation at finite budgets and a finite-penalty objective whose weight derivative generally differs from the feedforward loss gradient.

For ePC, advantages include an existing implementation in FabricPC, a global error gradient that reaches all layers in one pass, and potentially fewer iterations. Disadvantages include a global reverse-mode computation and memory footprint, curvature-dependent step sizes, and the distinction between small-step BP-like behavior and equilibrium PC behavior. The local final weight rule does not make its error inference local.

For PC-ALM, advantages are persistent local credit after residuals shrink, the conditional exact BP endpoint, and the encouraging deep-narrow experiments. Disadvantages are additional dual state and tuning, stability/finite-budget limitations, and untested library integration. A hard-constraint forward endpoint may be inappropriate if the goal is the original soft latent-variable model. All three use derivatives of local prediction functions. ALM does not remove the need for appropriate transposed Jacobian actions.

The terms ePC error and ALM multiplier must not be conflated. The first parameterizes a current discrepancy; the second accumulates constraint violations over inference time. At the ALM endpoint the discrepancy can vanish while the multiplier remains nonzero.

Transition

That distinction tells us which shared FabricPC contracts must change.

Sources

[P] Paper v1 · [I] sPC solver · [R] ePC solver · [T] ePC tests

17 FabricPC needs a coordinated extension

23:00–24:30

Talk track

There are four connected changes. First, store a per-example hidden multiplier in the inference state with a clear reset lifecycle. Second, add the augmented local term to the objective so both activity gradients and weight gradients see it. Third, implement the exact primal-residual-dual ordering and final primal-only step. Fourth, add diagnostics that distinguish prediction error, multiplier, composite credit, and stationarity.

This is more than overriding a latent-update formula. FabricPC's learning path recomputes local node derivatives, and its error field already has a precise meaning. The shared state, inference, and learning interfaces need to carry the same mathematical objective. Start with a small acyclic Gaussian network, then extend the common contracts to richer nodes once the gradients check out.

From first principles / preparation

Repository evidence: NodeState in core/types.py currently has z_{latent} , z_{mu} , error, energy, and latent_{grad}; it has no multiplier field. The state is a JAX pytree. Any chosen dual-state representation must be handled by state initialization, copying/replacement, batching, sharding, and pytree registration. Keep λ separate from error, because node pairing defines $\text{error} = z_{\text{latent}} - z_{\text{mu}}$.

InferenceBase in core/inference.py exposes inference_{step}, begin_{segment}, finalize_{state}, and run_{inference}. Its default step forms local activity gradients and then updates states. ALM needs control of the larger phase structure, including updated-state residuals and the final dual omission. An InferenceSchedule can compose solvers, but composing an ePC warm start with ALM would be a new algorithm, not a replication of the paper's forward initialization.

compute_{localweightgradients} in core/learning.py calls each node's forward_{andweightgrads}, so a modified signal stored only in a final error buffer would be recomputed away. The local augmented objective must feed this derivative path as well as inference. Preserve the trainer's established batch normalization: the low-level routine produces batch sums and the training wrapper normalizes them. Do not normalize twice.

The initial proposed scope is hidden Gaussian consistency constraints plus a separate supervised readout loss, with plain, explicitly matched scaling. Audit μ PC forward scales and gradient preconditioners before adopting paper hyperparameters. Arbitrary node energies, cyclic graphs, extra clamps, and a cross-entropy head require deliberate objective semantics. Scope expansion should update shared infrastructure and all affected callers; a special node that bypasses the shared energy contract would hide the inconsistency. No exact new class/API name is claimed to exist.

Transition

We can turn the proposal into a sequence of falsifiable tests.

Sources

[N] State types · [I] sPC solver · [W] Local learning · [R] ePC solver

18 Evaluate correctness before model-scale claims

24:30–26:00

Talk track

First, reproduce a scalar and a small linear graph, where the exact gradients are available. Verify that zero dual rate recovers the matched sPC trajectory and that a settled stable ALM solve matches BP. Test update ordering and per-example dual resets directly.

Next, compare BP, sPC, ePC in both of its relevant regimes, and ALM on the same small networks. Record gradient direction and magnitude, constraint and stationarity residuals, wall time, and memory. Then reproduce the paper's deep-narrow setting and extend to longer training and a CIFAR convolutional model. Promote the method only if it solves a measured problem our existing baselines do not already solve at comparable cost.

From first principles / preparation

Stage 1 is algebraic correctness, not a training contest. Use a scalar oracle and randomized linear acyclic networks with known BP gradients. Under $\alpha=0$ and λ initialized to zero, match sPC's objective, activity steps, initialization, precision, and scaling exactly. $T=1$ must omit the dual update. For stable positive α , convergence should drive both constraint residuals and activity stationarity toward zero and make the full weight-gradient relative error vanish. Test an independent hand-derived objective, not just the same helper called through two interfaces. These are proposed FabricPC checks; this package independently verifies only its scalar teaching example.

Stage 2 compares algorithms fairly. Report $\cosine(g_{\text{method}}, g_{\text{BP}})$, relative gradient error $\|g_{\text{method}} - g_{\text{BP}}\| / (\|g_{\text{BP}}\| + \text{small numerical guard})$, per-layer magnitudes, supervised loss, validation accuracy, time per update, time to target accuracy, and peak accelerator memory. Here g denotes the vector of all parameter gradients. Cosine can be excellent while the magnitude is wrong. Both algorithms must use the same weights, sample, and loss when comparing gradients. Use stable rate searches for each algorithm and comparable tuning effort, rather than sharing a numerical η that acts in different coordinates.

Stage 3 reproduces the paper protocol, then changes one axis at a time: more epochs, nonlinearity, width/depth, different initialization/scaling, precision, readout loss, and convolutional nodes. Compare both equal activity counts and equal measured compute budgets. Exclude compilation and warmup from steady-state timing; report them separately if startup matters. Run multiple seeds and state uncertainty. At $L=64$, the paper's $T=2L$ condition means 128 activity steps, so practical cost cannot be assumed small.

For an ePC/ALM hybrid, explicitly budget the global reverse passes and test the state/dual initialization at the handoff. A hybrid may be a useful later experiment, but it no longer tests wholly local inference. Go/no-go thresholds should reflect the team's objective: local credit fidelity, time-to-accuracy, or memory, rather than a universal percentage invented for this talk.

Transition

End with a decision the team can discuss.

Sources

[P] [Paper v1](#) · [D] [Stability report](#) · [T] [ePC tests](#)

19 Recommendation: test PC-ALM as a research method

26:00–27:00

Talk track

I recommend a bounded research implementation and a controlled comparison, rather than assuming the paper justifies changing our default training method. The mechanism is attractive because it changes what an equilibrated local network computes: the residual can disappear while the multiplier preserves the BP learning signal.

The critical comparison for FabricPC is with the ePC regime we actually use, at measured cost and over meaningful training durations. If our primary aim is studying local credit assignment, the paper gives a strong reason to proceed. If our primary aim is GPU training speed, we need evidence from the implementation before calling this an improvement.

From first principles / preparation

This is a recommendation, not a result from the paper. The requested deliverable is a presentation, so this package does not modify FabricPC, launch a large training campaign, or assert that a new solver is ready. The integration design is concrete enough to scope the next engineering discussion.

A useful first deliverable for an implementation project would be a correct acyclic linear/Gaussian ALM solver in the shared inference/objective framework, with independent gradient checks, explicit state lifecycle, and a comparison harness that already includes BP and ePC. Follow it with the published residual-MLP setting. Extending to the full arbitrary-graph/custom-energy surface of FabricPC at once would make it harder to tell whether failures came from the proposed method or altered objective semantics.

The three takeaways to emphasize are: sPC and ePC generally optimize the relaxed PC objective using different coordinates; ALM introduces dual accumulation to target the forward constraints; and reaching a BP endpoint is conditional on convergence, while practical value requires fair cost and stability measurements. The possible benefit is algorithmic and architectural. It is not established by the word augmented or by one favorable dataset grid.

Transition

Invite the team to choose the experiment's success criterion.

Sources

[P] [Paper v1](#) · [R] [ePC solver](#)

20 Discussion: what should the first experiment prove?

27:00–30:00

Talk track

I would like the team to decide which question matters most. Do we want to show that local dynamics can reproduce BP credit? Do we want better deep-network learning than our present PC configuration? Or do we want a practical improvement in training time or memory over ePC? Those lead to different first experiments and different definitions of success.

My proposed starting point is a correct linear acyclic implementation, followed by one matched deep-narrow comparison with BP, sPC, and both small-step and near-equilibrium ePC. From there we can decide whether longer training and richer nodes are justified.

From first principles / preparation

Reserve these three minutes for discussion rather than using them to deliver more prepared material. Backup slides follow, so stop here when presenting unless a question calls for one.

Likely question: Is this just backpropagation? Answer: the feasible stationary endpoint computes the BP weight gradient, but the procedure for obtaining it is a finite sequence of adjacent-layer primal-dual updates. At finite inference it can differ. Ordinary backprop computes the exact composed derivative directly in a reverse pass.

Likely question: Is the multiplier the same as an ePC error? Answer: no. The ePC error is an optimization coordinate that reconstructs the current hidden state. The multiplier is accumulated constraint pressure. At an ALM endpoint, residuals vanish but multipliers need not.

Likely question: Why not raise the PC penalty? Answer: it tightens the soft constraints but increases curvature and can restrict numerical step sizes. ALM can enforce feasibility at finite penalty under a convergent solve, at the cost of dual state and coupled dynamics.

Likely question: Does this work with cycles, attention, or cross-entropy? Answer: some extensions have plausible local formulations, but the paper's theorem and experiments do not establish them. A differentiable supervised loss can be substituted in the KKT algebra; stability and practical performance still need analysis and testing. Cycles also change the meaning and uniqueness of the forward solution.

Likely question: Should we combine ePC and ALM? Answer: possibly as a separate measured experiment. Warm-starting ALM from ePC changes the published initialization, and ePC uses global differentiation, so any locality claim must state that. Do not accidentally present a hybrid result as the paper's algorithm.

21 Notation and conventions

Optional backup

Talk track

Use this slide to resolve notation rather than to add new claims. In particular, the prediction residual, ePC error coordinate, ALM multiplier, and BP derivative are distinct roles. The sign convention is state minus prediction throughout.

From first principles / preparation

Recurring symbols: x is the input; y is the target; $\hat{y}$ is the model's readout prediction; L counts weight-bearing layers; N is uniform hidden width when discussing the experiments; i indexes a hidden layer; W_i is its weight matrix; θ collects all weights; h_i is its activation and $h_0=x$; σ is the elementwise activation; σ' is its derivative; J_{i+1} is the adjacent layer's derivative with respect to h_i ; ℓ is the supervised loss; F_{PC} is the PC energy; L_ρ is the augmented Lagrangian.

$r_i=h_i-\sigma(W_i h_{i-1})$ is the hidden constraint residual. ε_i is the optimized error coordinate in ePC; it equals that residual when the derived state is consistent. λ_i is the ALM multiplier for the hidden constraint. $c_i=\lambda_i+\rho r_i$ is composite credit. $\delta_i=dl/dh_i$ is the BP activation derivative at the forward state. At a feasible stationary ALM endpoint, $\lambda_i=-\delta_i$. Never use λ alone to mean a curvature eigenvalue in this presentation; that scalar is called κ in the backup stability material.

ρ is the hidden penalty strength; α is the dual ascent rate; η_h is the state descent rate; η_ε is the ePC error-coordinate rate; η_θ is the weight learning rate; T counts activity steps in the paper's Algorithm 1. A star denotes a settled value. The transpose superscript T in a matrix expression is unrelated to the step count T . The norm is Euclidean for vectors and Frobenius for a weight matrix. The Hadamard product $\odot$ multiplies components elementwise.

In the linear stability appendix only, A maps stacked hidden activations to the linear hidden constraints, b contains the input-dependent offsets, C maps stacked hidden activations to the output readout, $B=C^TC$ is supervised activity curvature, and M is the joint primal-dual iteration matrix. These are separate from a minibatch, which is always spelled out here.

Sources

[P] Paper v1 · [R] ePC solver

22 Derive the scalar example by hand

Optional backup

Talk track

Differentiate the PC energy with respect to h and set it to zero to get $h=0.6$. For ALM, the fixed dual update requires $h=0.2$; substituting that into the stationary activity derivative gives $\lambda=0.8$. The corresponding local weight gradients then reproduce the two BP derivatives.

From first principles / preparation

For general scalar x , y , w_1 , w_2 , and penalty ρ , define $F_{PC}(h) = \frac{1}{2}(y - w_2 h)^2 + (\rho/2)(h - w_1 x)^2$. Its derivative is $-w_2(y - w_2 h) + \rho(h - w_1 x)$. Setting it to zero gives $h_{PC} = (w_2 y + \rho w_1 x) / (w_2^2 + \rho)$. Its curvature $w_2^2 + \rho$ is positive, so the solution is unique.

For ALM add $\lambda(h - w_1 x)$. A fixed point with $\alpha > 0$ requires $h_{ALM} = w_1 x$. Stationarity then gives $\lambda_{ALM} = w_2(y - w_2 w_1 x)$. The weight gradients at that endpoint are $\partial L_{\rho} / \partial w_1 = -\lambda_{ALM} x$ and $\partial L_{\rho} / \partial w_2 = -(y - w_2 w_1 x) w_1 x$. Directly differentiating the composed loss $\frac{1}{2}(y - w_2 w_1 x)^2$ yields the same two values.

At our numerical values, PC's relaxed energy is 0.16 and the feedforward loss is 0.32. Those are different objective values at different states, so the smaller relaxed energy is not evidence of a better forward prediction. The forward output remains 0.2 until the weights are actually updated. At ALM feasibility the hidden energy contribution vanishes and the augmented objective equals the forward loss, while the multiplier still affects the derivative taken with respect to unconstrained local variables.

For our selected rates, the linear error-state recurrence has matrix $[[0.6, -0.2], [0.3, 0.9]]$ around the fixed point. Its eigenvalues have magnitude $\sqrt{0.6} \approx 0.7746$, below one, so the teaching trace converges. The code checks the endpoint gradients and a finite Algorithm 1 schedule independently. In one dimension, ePC's $h=0.2+\epsilon$ transform is a translation, so its energy Hessian and equally stepped trajectory are the same as sPC's.

Sources

[P] Paper v1

23 Why ePC changes dynamics but preserves the objective

Optional backup

Talk track

The error-to-state transformation has an invertible triangular Jacobian. That preserves stationary points of the energy. But gradient descent depends on coordinates. An error-space gradient step produces a state-space update shaped by the Jacobian times its transpose, which couples distant layers. This accounts for faster signal access without changing the relaxed objective.

From first principles / preparation

For this backup derivation, define $Q = \partial h / \partial \epsilon$, the full Jacobian from stacked free errors to stacked free hidden states on a directed acyclic graph. The recurrence $h_i = \sigma(W_i h_{i-1}) + \epsilon_i$ makes Q block lower triangular with identity diagonal blocks. Therefore Q is invertible even when an individual W_i is singular.

The chain rule gives $\nabla_{\epsilon E} = Q^T \nabla_{h FPC}$. Thus $\nabla_{\epsilon E} = 0$ if and only if $\nabla_{h FPC} = 0$. At a stationary point, the Hessians satisfy $H_{\epsilon} = Q^T H_{hQ}$, where $H_h = \nabla^2_{h FPC}$ and $H_{\epsilon} = \nabla^2_{\epsilon E}$. Terms involving second derivatives of the coordinate map vanish there because $\nabla_{h FPC} = 0$. This congruence preserves the numbers of positive and negative curvature directions, but generally changes eigenvalues and conditioning.

For a small error gradient step, $\Delta h \approx Q \Delta \epsilon = -\eta_{\epsilon Q Q}^T \nabla_{h FPC}$. Because Q includes downstream dependence, $Q Q^T$ can couple nonadjacent states. The energy has not gained a dual constraint term; its optimization geometry changed. A finite step in a nonlinear transformation additionally has higher-order terms.

Near a positive quadratic mode of curvature κ , a gradient step multiplies its distance to equilibrium by $1 - \eta_{\epsilon \kappa}$. After T steps the relaxed fraction is $1 - (1 - \eta_{\epsilon \kappa})^T$. This is exact for fixed quadratic dynamics and a local approximation otherwise. Negative curvature modes do not represent relaxation toward a minimum. The repository measures these properties, but the thresholds used to label ‘BP-like’ or ‘near PC equilibrium’ are descriptive choices, not universal mathematical phase boundaries.

Sources

[R] ePC solver · [T] ePC tests · [D] Stability report

24 Linear stability is a property of the joint update

Optional backup

Talk track

The full theorem requires all eigenvalues of the joint activity-and-multiplier iteration to lie strictly inside the unit circle. The frequently quoted scalar bound is derived after dropping the supervised-loss curvature. It is useful intuition, but it should not be used as a universal safe hyperparameter rule for a nonlinear FabricPC model.

From first principles / preparation

Stack all hidden activations into h and write linear residuals as $r=Ah+b$. A is block lower bidiagonal for a chain, with identity blocks on the diagonal and negative adjacent linear maps below them; b holds the clamped-input offsets. Let C map h to the readout prediction and $B=C^TC$. B is positive semidefinite and represents the supervised squared-loss curvature with respect to hidden states.

Define $K=I-\eta_h(B+\rho A^TA)$. A primal step maps a perturbation of h and λ to $K\Delta h-\eta_{hA}^T\Delta\lambda$. The following dual update uses the NEW primal state, giving joint matrix $M=[[K,-\eta_{hA}^T],[\alpha AK,I-\alpha\eta_{hAA}^T]]$. The spectral radius $\text{spr}(M)$ is the largest absolute eigenvalue. For the fixed linear system, positive ρ , α , and η_h with $\text{spr}(M)<1$ imply convergence to the unique feasible stationary endpoint. This is not a global nonlinear convergence result.

If $B=0$, diagonalizing by singular modes of A gives the condition $\eta_h s^2(2\rho+\alpha)<4$ for every singular value s , with positive rates and a full-rank constraint operator. The corresponding PC condition is $\eta_h s^2\cdot 2\rho<4$. In our notation s is a singular value, avoiding conflict with the activation function σ . The simplified bound narrows as α rises. At $\alpha=0$ the primal dynamics reduce to PC with frozen zero multipliers; the entire joint-state matrix retains trivial constant dual modes, so one should not require strict joint contraction of unused dual variables in that special case.

The paper's approximate numerical caps depend on architecture and parameterization. Including B can change stability. With nonlinear maps, Jacobians and curvature change as activities and weights move. A linear spectral check is a diagnostic or local approximation in that setting, and finite-step oscillations can still damage training before a simplistic threshold flags divergence.

Sources

[P] [Paper v1](#)

25 Integration details that affect the algorithm

Optional backup

Talk track

The main engineering risks are mathematical contract mismatches. Multipliers must be per example and must reach both local derivative paths. The dual step must read new-state residuals. The output loss must remain a loss. Scaling, custom energies, and cyclic schedules must be audited before claiming the paper's guarantees apply.

From first principles / preparation

A conceptual step is: compute each hidden prediction at old states; form the local augmented terms with old multipliers; accumulate all adjacent activity derivatives; update all free activities; recompute predictions and true residuals at new states; update the hidden multipliers unless this is the final primal step; after inference, recompute local augmented weight derivatives at final states. The input clamp and supervised target remain fixed during this process. All hidden multipliers reset at the next independent batch.

Sources and supervised heads have different roles from constrained hidden nodes. In a simple clamped-output Gaussian representation, the output node's mismatch is the supervised task loss and has no ALM multiplier. Internal hidden Gaussian errors are the constrained quantities. If a custom node energy includes an attractor or regularizer in addition to reconstruction error, the proposal must specify which part is a hard constraint and which remains an objective term. Multiplying every scalar node energy by a multiplier is not the augmented Lagrangian derived in the paper.

The NodeState extension must propagate through initialization, JAX pytrees, tracking snapshots, device sharding, and any state constructors. A batch-wide vector of multipliers would mix unrelated examples and is incorrect. A persistent multiplier across different samples would change Algorithm 1; any deliberate reuse would need a sample-indexing and stale-state argument.

Existing μ PC helpers scale inputs and gradients differently. Apply derivatives of the actual chosen objective and justify any preconditioner; otherwise KKT cancellation and the $\alpha=0$ equivalence check may fail. Convolutions and attention can in principle supply local vector-Jacobian products through their own forward map, but their internal activations, masks, normalization, and batch dependence need attention. Cyclic constraints can be written algebraically, but may have multiple or no feasible solutions and are not the paper's unique feedforward inner problem. Treat those as explicit later scope, with updated tests and claims.

Sources

[N] [State types](#) · [I] [sPC solver](#) · [W] [Local learning](#) · [T] [ePC tests](#)

26 Sources and reproducibility

Optional backup

Talk track

The links here point to the supplied paper and the exact FabricPC commit inspected. The separate guide includes the full derivations, source references, caveats, and proposed tests. The scalar plots can be regenerated from the included build script.

From first principles / preparation

Primary paper: Jeffrey Seely and Julian Gould, Augmented Lagrangian Predictive Coding, arXiv:2605.31022v1, 29 May 2026. Mechanism: equations 1–13 and Algorithm 1. Endpoint identity: Appendix A. Linear dynamics and conditional convergence: Appendix C. Wavefront approximation: Appendix D. Compute comparison: Figure 8 and Appendix E. Experimental scope: Appendix F. Limitations: Appendix G. Slides 13 and 14 reproduce Figures 3 and 2, respectively, from the supplied PDF.

Related ePC paper: Cédric Goemaere, Gaspard Oliviers, Rafal Bogacz, and Thomas Demeester, ePC: Fast and Deep Predictive Coding in Digital Simulation, arXiv:2505.20137. The work previously appeared under an Error Optimization title. Our technical implementation claims are grounded in FabricPC source rather than assuming every historical paper version uses the current library semantics.

FabricPC source snapshot: commit 8406e6a838442391fd3089958e1a2c1b44c57eda, retrieved 22 September 2026. The solver's global error differentiation is in `core/inferenceepc.py`. The sPC lifecycle is in `core/inference.py`. Dynamic state and pytree registrations are in `core/types.py`. The local learning path is in `core/learning.py`. The ePC user guide describes regime measurement, and the report and tests document long-run observations and equivalence qualifications. These are code/document inspection findings; they were not re-benchmarked here.

The presentation build checks the scalar endpoint calculations, schedule length, slide bounds, note separation, and document output. The paper's plots are reproduced as labeled source figures rather than redrawn from guessed data. Proposed FabricPC class structure, tests, and experiments remain proposals. The complete private notes are embedded in the PowerPoint notes part and also supplied separately; the audience PDF contains only slides.

Sources

[P] Paper v1 · [E] ePC paper · [R] ePC solver · [I] sPC solver · [N] State types · [W] Local learning · [G] ePC guide · [D] Stability report · [T] ePC tests